Formal Equivalence of Hazard3 and the RISC-V Sail Model

An unbounded, open-tool proof of an RV32IMC configuration of the RP2350 core, modulo one stated axiom, and the six divergences it found

Leo Strijbos Tom de Vrieze

September 2026

Abstract

Hazard3 is the RISC-V core in the Raspberry Pi RP2350. We port the lowRISC/Oxford Ibex proof harness to Hazard3 and prove, with Yosys and the rIC3 model checker, that an RV32IMC + Zicsr machine-mode-only build of Hazard3 v1.1.1 is trace equivalent to the RISC-V Sail specification: at every specification query the program counter, the six checked CSRs and the register operands agree, and the core's memory requests are the specification's memory requests, for every instruction sequence and without a depth bound. All 18 top-level goals close. The proof rests on 265 helper lemmas in 62 layers and on one axiom, the arithmetic result of the sequential multiplier/divider, which the Ibex proof also assumes. Six divergences had to be fixed before the proof could close: five in the Hazard3 RTL (reserved `C.ADDI4SPN` encodings execute, `mstatus.MPRV` writable without U-mode, a trapping `meicontext` write with a side effect, `FENCE` trapping on non-zero rs1/rd, and vectored trap entry using OR rather than addition) and one in the lowRISC revision of the Sail model (`mstatus.TW` not masked without U-mode). All five RTL divergences reproduce on Hazard3's own simulator; one had been fixed independently on the development branch before we found it. The lemma discovery and the proof runs were done by an LLM agent under human direction; we report what that required.

1 Introduction

A processor is conformant when it does what the ISA says for every instruction sequence. Testing samples that space; a formal equivalence proof against an executable specification covers it. The RISC-V Sail model is such a specification, and the lowRISC and Oxford proof of the Ibex core against it [1] showed that an unbounded equivalence proof of a small RISC-V pipeline is within reach of a small team with open tools.

This report applies that method to Hazard3 [2], Luke Wren's three-stage RV32 core, which is the RISC-V option in the Raspberry Pi RP2350. Hazard3 differs from Ibex in ways that matter to a proof: an AHB5 bus with a single data port, a compressed-instruction buffer with a lock that pins the decode stage across a predicted branch, a sequential 32-cycle multiplier and divider, and an interrupt controller of its own. None of the Ibex lemmas transfer as written. What transfers is the shape of the proof and the discipline around it.

1.1 What is claimed

Let S be the specification state, A the micro-architectural state of Hazard3, $abs : A \to S$ the abstraction that reads the architectural registers out of the pipeline, and let $Spec$ and $H3$ be the one-step functions of the specification and of the core, with $SpecOut$ and $H3Out$ their memory operations. The 18 goals establish two things.

- **Wrap**: $Spec(abs(a), i) = abs(H3(a, i))$ for every reachable a and instruction i. One step of the specification and one step of Hazard3 land in the same architectural state.
- **Load, Store, NoMem, Mem**: $SpecOut(abs(a), i) = H3Out(a, i)$. The memory operation Hazard3 performs is the one the specification performs, with the same address, direction, byte lanes and data.

By induction over queries, the nth memory operation agrees for every n, and the architectural state agrees at every query, from a common start. The claim is conditional on instructions continuing to retire: liveness is not proven, the same caveat the default Ibex flow carries.

The proof is of a patched copy of the RTL. Five one-line changes fix the five Hazard3 divergences of section 5; without them, the `Wrap` goals for mstatus, mcause, mepc and the PC are false. The specification is the lowRISC revision of the Sail model with three patches of ours, one of which fixes a defect in that revision. Every claim in this report is therefore about Hazard3 v1.1.1 with those five changes, in the configuration of section 2.1, under the assumptions of section 7.

1.2 Summary of results

- All 18 goals close, each with the verdict *proven modulo axioms [CtiWbMulDiv]* from a chain checker that walks the proof log and requires every constraint used at proof time to be itself proven on the same model.
- One axiom, stated in full in the proof file: for the eight RV32M instructions, the value the datapath produces equals the specification's value. Everything around it (that the instruction retires, writes only rd, does so at the right time, does not trap, advances the PC) is proven.
- Six divergences, each a machine-checked counterexample found by bounded model checking from reset, each reproduced independently of the proof on Hazard3's CXXRTL testbench with a program of a few dozen hand-encoded instructions.
- The lemma loop ran from 17 to 20 September 2026 on one 48-core machine, in 44 rounds, and tried 258 distinct lemma candidates, of which 14 were refuted for real. The whole proof was then re-run from the pinned sources on a fresh machine, with the same 18 verdicts.

2 Setup

2.1 The device under test

Hazard3 at tag v1.1.1, commit `8af99293`, the head of its `stable` branch. The configuration is Hazard3's minimum that still has a multiplier and compressed instructions: RV32IMC + Zicsr, `MULDIV_UNROLL = 1` (the sequential shift-add multiplier and restoring divider, not the fast multiplier), no A, no bit-manipulation, no Zcb/Zcmp, no Zifencei, no Xh3 extensions, machine mode only, no PMP, no debug, no branch predictor, one external interrupt line plus the software and timer interrupts. The RP2350 ships a larger configuration (RV32IMAC with Zba, Zbb, Zbs, Zbkb, Zcb, Zcmp, the Xh3 extensions, U-mode, PMP and debug). What is proven here is a subset of that core's behaviour, not the RP2350's build.

The RTL is read by Yosys through yosys-slang. Two kinds of patch are applied to a copy, and a manifest is written with every build. *Tool patches* change no behaviour (two functions are marked `automatic` so the front end accepts them). *Conformance patches*, applied only with

`H3_CONFORMANCE=1`, change behaviour and are the fixes for the five findings. A proof of the tool-patched design is a proof of the shipped design; a proof of the conformance-patched design is a proof that the fixes close the divergences.

2.2 The specification

The Sail model is lowRISC's fork of `riscv/sail-riscv` at commit `81a266b6`, the revision Ibex's formal flow pins, compiled to SystemVerilog by lowRISC's Sail back end. Three patches of ours are applied.

- **Gate 6** undoes two Ibex specialisations that are not architectural: lowRISC had replaced upstream's WARL legalisation of `mtvec` with a 256-byte-aligned, vectored-only policy and added an `mcause` legalisation carrying Ibex's internal interrupt encoding; and its memory rework had deleted the call to `check_misaligned`, so the model split every misaligned access regardless of the platform flag. Both are restored to upstream's behaviour, parameterised so that the Ibex policy is one setting of the parameter.
- **Gate 7** makes `FENCE.I` decode conditional on Zifencei, which this configuration of Hazard3 does not implement.
- **Gate 8** masks `mstatus.TW` when U-mode is absent. This is finding 6: a defect in the lowRISC revision, fixed upstream in February 2025.

2.3 Tools

Yosys with yosys-slang for the RTL, lowRISC's Sail back end for the specification, `psgen` for the proof script, `aig-manip` (lowRISC, Rust) to select a property and its constraint set out of the AIGER model, and rIC3 for bounded model checking, k-induction and IC3. All come from the Nix flake of the Ibex formal flow at Ibex commit `405c6d1d`. The proof machine was one AWS r7i.12xlarge (48 vCPUs, 384 GB), running 18 proofs in parallel.

The full model, `all.aig`, has 10.4 million AND gates and 4762 latches; the specification appears as a combinational block with 1.8 million free inputs. A second model, `core.aig`, is the same harness with the specification's outputs left as free inputs. It is built from the identical property set, and is the basis of the pre-screen of section 3.3.

2.4 The harness

The harness is a port of Ibex's `dv/formal`. Its parts, in the order they were built and validated (the bring-up notes in `docs/BRINGUP.md` record each gate with its measurements):

1. **Bus environment.** Hazard3's own AHB5 slave assumptions with unbounded stalls, and its AHB5 master properties as proof obligations. A negative control (a slave that violates the protocol) was checked to be detected.
2. **Free state and abstraction.** Registers that reset to unknown values become free inputs; the abstraction function reads the six checked CSRs, the PC and the register file out of the pipeline at the commit point.
3. **The commit point.** The cycle at which an instruction's architectural effect is fixed, with the proof that every architectural change (PC, CSRs, register file) happens at a commit and has a cause.

4. **The follower and the ghost.** The follower is a small record, carried alongside the pipeline, of what the specification was told at the last query and what it answered. The ghost is the specification's predicted next state, kept until the next query and dropped when the harness declines to compare (bus faults, CSRs outside the allowlist). Ibex's obligations on the specification's answer become implications guarded on the follower.

5. **The goals.** The 18 goals of table 1, stated so that each fails, if it fails, for a reason a counterexample shows. Negative controls prove they are not vacuous: a query does occur with the ghost valid, an instruction does read a register the ghost knows, memory requests do happen and do not always happen.

6. **The lemma loop.** Section 3.

Every property in the proof script mentions only signals inside the core and on the bus. The specification enters only through the follower and ghost, which is what allows the same property set to be built into both models.

3 Method

3.1 The lemma staircase

Each property in the proof script sits in a layer. A property is proven by k-induction with every property in the layers beneath it as a constraint, and nothing above or beside it. `aig-manip select` extracts, from the full model, the property and exactly that constraint set; rIC3 proves it or returns a counterexample to induction. The final proof has 62 layers, 265 helper lemmas, 18 goals and 1 axiom. The chain for `WrapPc`, the last goal to close, uses all 266 (265 lemmas and the axiom): 91 spec-free lemmas in the base layer, from the bring-up, and 174 discovered in the loop.

Lemmas in the same layer cannot see each other. This bit twice: a pair of lemmas that each needed the other would time out together and prove instantly once split across two layers. The runner now inserts every candidate in its own layer unless told otherwise.

3.2 Counterexamples to induction as the source of lemmas

k-induction fails on a state that satisfies all constraints and the property at step k and violates it at step $k+1$. That state is usually unreachable, and the lemma to write is the fact that rules it out. The runner produces for every candidate, in parallel with the proof attempt, a counterexample to induction at $k = 1$ (`aigicti.py` builds the inductive-step model, rIC3 finds the trace, `aig-manip simulate` turns it into a VCD), and a report listing which harness predicates are false in the final frame and which pipeline signals changed between frames. Reading that report is the whole of lemma discovery. A typical cycle: the report shows the decode stage holding an instruction with a register operand read from a stale port while the pipeline is locked; the lemma says that under a lock the operand comes from the forwarding network; that lemma proves in 200 s; the target proves in the next round.

The $k = 1$ counterexample is not a predictor of failure. Sixteen candidates with a $k = 1$ counterexample were proven by k-induction at depth 1 in the same round, because rIC3's k-induction strengthens with the constraints in ways the bare inductive step does not. Early versions of the runner skipped the proof when a counterexample existed and lost those sixteen; the current one always runs both.

3.3 Two models

Roughly half of all lemmas are about the pipeline alone and never mention the follower. Those prove on `core.aig`, where the specification is free, and a proof there transfers to `all.aig` because the models differ only in what drives the specification's outputs. Every candidate is first tried on the core model (bounded check to depth 15, then k-induction with a 400 s cap); a proof is recorded and the candidate skips the full model, a refutation on the core model is disregarded (the free specification can refute anything that depends on it), and a timeout decides nothing. On the full model each candidate gets a bounded check and then k-induction with a 600 s cap. The caps came from the data: the median proof took 164 s, the 90th percentile 265 s, the longest 462 s, and a candidate that had not proven by then ran to whatever cap it was given. The full-model cap was raised from 400 to 600 s after the 462 s proof.

Of the structural lemmas from the bring-up, 48 do not prove by k-induction on the full model at any cap tried; after each model change they were re-proven with IC3 (1800 s budget, all within it) instead.

3.4 Chain of custody

The proof script and the log together define what is proven. A lemma is proven at a moment, against the constraint set present at that moment; if a constraint beneath it is later refuted, removed or found never to have been proven, the lemma is stale. On 18 September two goals were announced proven and retracted the same day. One rested on a lemma that had only proven because of an unproven lemma beneath it. The other had sixteen leftover instrumentation properties as constraints, five of them vacuity controls written to be false ("an ADD never retires"), so every proof above them had assumed five instruction classes never execute. Bounded model checking cannot flag a false constraint; it silently shrinks the explored space.

The chain checker, `tools/chain_check3.py`, was written that day. For a goal it reconstructs, from the insert and remove events in the log, the constraint set present at the goal's proof line; checks that each constraint is still present and has itself a valid proof, recursively; treats the structural lemmas as taken on the proof header's word and says so; and prints *PROVEN*, *PROVEN MODULO AXIOMS [names]* if an assumed lemma is in the chain, or *STALE* with the offending constraint. Two weaker checkers (present and has an UNSAT line; a count of constraints) both passed states that the third called stale. No goal has been reported since without that output, and the rule the work ran under was that no report of a goal is made without it.

Two consequences of the discipline. Removing a lemma means re-proving every survivor above the lowest removed layer, which is what made stripping a bad lemma expensive and made the runner's strip step re-prove only survivors, not everything. And a model change (a conformance patch) means rotating the log and re-proving every lemma from nothing; it was done twice, on 19 September (four RTL fixes and the Sail fix) and 20 September (the `mtvec` fix), each time as a full sweep with nothing stripped, followed by the chain check on all 18 goals.

3.5 The runner

`tools/h3round_v41.sh` runs one round: for each candidate lemma in a list, insert it in the proof script at a stated layer, rebuild the select file, try the core model, then the full model with the counterexample analysis in parallel, and append `RESULT` lines to the log. Candidates can be tagged `.dut` (skip the core model; the lemma mentions the follower) or `.axiom` (insert, never attempt, log `AXIOM name assumed`). Goals are tried after the candidates: a bounded check to depth 15, and

k-induction only if no counterexample to induction exists. A round of 5 to 15 candidates took 15 to 70 minutes. Nothing in the runner kills a running proof.

4 Results

4.1 Goals

goal	statement
WrapPc	the PC agrees at each query
WrapMstatus, WrapMie, WrapMcause, WrapMtvec, WrapMscratch, WrapMepc	the six checked CSRs agree at each query
WrapRegA, WrapRegB	both register read operands agree at each query
LoadEn, LoadAddr	a load is issued when and where the spec loads
StoreEn, StoreAddr, StoreBe, StoreWData	a store is issued when, where, which lanes, which data
NoMem, MemWe	no request without a spec memory op; direction agrees
MemNoSecondGranule	no access is split into a second granule

Table 1: The 18 goals. Every verdict on the final model is *proven modulo axioms [CtiWbMulDiv]*. `MemNoSecondGranule` closed first, on 18 September; `WrapPc` closed last, on 20 September, and the sweep on the fully patched model re-proved everything and re-checked all 18 chains the same day (`results/chain/chain_all.txt`).

4.2 The axiom

`CtiWbMulDiv`: whenever the M stage holds an instruction whose follower record is clean (valid, not excepting, not a fetch fault, one the specification could execute) and whose word is an RV32M operation (opcode OP, funct7 `0000001`: MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, REMU) and the specification writes rd, then the value the core holds for that write over its M residency equals the specification's value.

What is assumed is only the arithmetic of the iterative unit for those eight instructions. What is proven above it is that such an instruction retires, writes rd and only rd, at the right time, does not trap, advances the PC, and that every other instruction class is value-correct: 39 per-class lemmas (`CtiWbAdd` through `CtiWbCXor`), the load lemma, the CSR lemma and their union `CtiWbAll`.

We tried to discharge it. A bounded check of the multiplier/divider unit alone against a plain 64-bit product and 32-bit quotient found no counterexample to depth 34 for MUL, MULHU and DIVU and stalled there (30 minutes per depth); k-induction and IC3 do not close a 32-cycle iterative multiplier at the bit level. The algebraic route (AMulet-style circuit verification against a polynomial specification) was estimated at two to three days for the multiply half only, with division still assumed, and was not taken. The Ibex proof carries the same assumption as `MType`. Hazard3's own riscv-formal checks cover the unit in bounded form upstream.

4.3 Statistics

5 Findings

Each finding started as a bounded-model-checking refutation of a goal or lemma from reset, was traced to an RTL or Sail line, was fixed by a conformance patch, and later was reproduced outside

helper lemmas in the final proof	265
of which found in the loop (named `Cti*`)	174
of which spec-free, in the base layer, from the bring-up	91
layers	62
constraints in the `WrapPc` chain	266
rounds (17 to 20 September)	44
distinct lemma candidates attempted	258
refuted for real (the candidate was false)	14
candidates that timed out at least once	110
k-induction proof time on the full model, median / p90 / max	164 / 265 / 462 s
model changes with full re-proof	2

Table 2: Proof-run statistics from the three log files.

the proof with a program of a few dozen instructions on Hazard3's own CXXRTL testbench at v1.1.1 (`findings/upstream/repro/`). Each program exits 1 on v1.1.1 and 0 on v1.1.1 with the patch applied; the same programs were run on the `develop` head of 29 August 2026 (`ba0c83c6`). Hazard3's contribution guide asks that reports contain no RTL suggestions, so the reports describe behaviour only. Findings 1, 2, 3 and 5 were filed on 21 September 2026 as Hazard3 issues 44 to 47.

1. Reserved C.ADDI4SPN encodings execute. `C.ADDI4SPN` has $nzuimm = instr[12{:}5]$ and the encodings with $nzuimm = 0$ are reserved. The decompressor tests `instr[12:2]`, which also requires $rd' = 0$, so only the all-zero halfword traps; the other seven encodings (`0x0004` to `0x001c`) retire as `addi rd', sp, 0`. Every sibling reserved test in the file checks exactly the field the specification names. Present on `develop`.

2. mstatus.MPRV writable without U-mode. With `U_MODE = 0` the privileged specification makes MPRV read-only 0. The mstatus write arm gates MPP and TW on `U_MODE` but not MPRV, which sets and reads back (mstatus `0x00021800` after `csrs mstatus, 0x20000`). The bit feeds the effective load/store privilege. Present on `develop`.

3. A trapping meicontext write modifies mie. With `EXTENSION_XH3IRQ = 0` the CSR does not decode and a write raises illegal-instruction, but the arm that applies the write's side effect to `mie` is gated only on the address, so `mie` goes from 0 to 8 in the trap-entry cycle. An instruction that raises an exception must not change architectural state. Present on `develop`.

4. FENCE with non-zero rs1 or rd traps. The opcode pattern pinned rs1 and rd to zero; the unprivileged specification says base implementations shall ignore those fields. Widening the pattern alone would make `fence` with rd = x4 write x4, so the decode arm must also force rd to x0. The maintainer fixed this on `develop` on 22 August 2026 (`6adda670`) from a newer riscv-arch-test run, before this work found it on 19 September; the fix is not on `stable`. We record it as an independent rediscovery and do not file it.

5. Vectored trap entry uses OR. The specification sets $pc = BASE + 4 \cdot cause$; Hazard3 computes $BASE \mid 4 \cdot cause$ and documents that the table must be aligned to its size. That is a legitimate implementation choice if the register enforces it, but with the default `MTVEC_WMASK` every BASE bit is writable, so an unaligned BASE in vectored mode is stored, reads back, and sends a software interrupt (cause 3) to `BASE | 0xc`, four bytes before `BASE + 0xc`. The witness

was a nine-frame trace from the interrupt-entry lemma. Filed as a conformance question, since the documented alignment rule keeps ordinary software clear of it.

6. The Sail model lets TW be set without U-mode. `legalize_mstatus` in the lowRISC revision masks MPRV when U-mode is absent but not TW, so a `csrs` with bit 21 set leaves TW = 1 where Hazard3 correctly hardwires it. The same witness that showed finding 2 showed this (specification mstatus `0x00201888`, core `0x00021888`). Upstream `riscv/sail-riscv` fixed it in `8de2591` (19 February 2025); the lowRISC `ibex` branch head (`5fca3f99`, July 2025) still lacks it.

What the findings are not. None is reachable by compiler-generated code; 2 and 3 need non-default configurations; 5 is documented. They are conformance divergences from the executable specification, of the kind that testing does not generate cases for (no test author writes a case for an instruction that should not exist) and that an equivalence proof produces as a by-product. Finding 4 shows the other route: a newer conformance suite that enumerates the encodings found it too.

6 Negative results and lessons

- **Lemma candidates are false about a third of the time at first writing, and the false ones are the informative ones.** Fourteen candidates were refuted outright. Each refutation was a real behaviour of the core the candidate had not allowed for: a load-use stall makes the forwarded operand differ from the register file for a cycle; an excepting instruction in the M stage leaves a dead slot whose rd matches the next instruction's rs1; a late data-phase bus fault turns a locked, non-excepting M stage into an excepting one. The operand-tie lemmas went through three versions before the guards were right.
- **The $k = 1$ counterexample does not predict k-induction failure.** Sixteen lemmas with a counterexample at $k = 1$ proved at depth 1 in the same round. A runner that skips the proof on a counterexample loses them.
- **Same-layer blindness.** Two lemmas that need each other and sit in one layer both time out. They proved at once when placed in consecutive layers.
- **Stale proofs after a strip.** A goal proven with lemma L beneath it stays "proven" in the log after L is stripped. The chain checker catches it; the fix is to strip and re-submit the dependents fresh, since re-inserting a name the checker has seen removed confuses its layer-order reconstruction.
- **Instrumentation is a constraint.** Probe properties left in the script after an experiment are constraints for everything above them, and a probe designed to be false makes every proof above it vacuous in the region it excludes. Strip them when the experiment ends, and re-prove.
- **k-induction with a 400 s cap is the wrong tool for the structural lemmas on the full model.** They prove with IC3 in under 30 minutes and time out under k-induction indefinitely. The model-change protocol runs IC3 on those 48 and k-induction on the rest.
- **The multiplier does not yield to bit-level SAT.** Bounded checking of the unit alone stalls at depth 34; the proof was not attempted further and the lemma is an axiom.
- **The yosys-slang front end sizes `{32'd0,a}*{32'd0,b}` as a 63-bit product.** Explicit 64-bit wires are needed for a checker that compares against a full product.

7 Scope and threats to validity

- **Configuration.** RV32IMC + Zicsr, machine mode only, sequential multiplier, no PMP, no debug, no branch predictor. The RP2350 configuration is larger in every one of those dimensions. Nothing here is a statement about the atomics, the bit-manipulation instructions, U-mode, PMP or the debug module.
- **The axiom.** The arithmetic of MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM and REMU is assumed. A wrong product would not be caught.
- **Patched RTL.** The proof is of v1.1.1 with five one-line changes. The shipped RTL is not equivalent to the specification; the findings say where.
- **Coverage narrowings.** Bus faults on either port are reachable but not compared: the Sail model has no channel for a bus error, so the faulting instruction and its trap are outside the comparison. CSR accesses outside the 16-address allowlist drop the ghost until the next query. `mtval` and `mcounteren` are fed as zero and not compared.
- **Assumptions.** Hazard3's AHB5 slave assumptions with unbounded stalls; debug port tied off (`DEBUG_SUPPORT = 0`); `mhartid` arbitrary but constant; and the bus returns the data the specification was given (`MemRdataIsSpecData`, Ibex's `mem.sv` assumption), a renaming of an otherwise unconstrained free input.
- **Not proven.** Liveness (that a query keeps happening), reset correctness, and instruction-fetch correctness (that the instruction executed at a PC was the one loaded from that PC). All three are open in the Ibex proof as well.
- **The specification.** A proof against a model is a proof against that model. We changed the model in three places; finding 6 shows the model can be wrong, and our own patches could be too. Gate 6 restores upstream behaviour and was checked to reproduce upstream bit-for-bit at the default parameter; Gate 8 matches upstream's later fix; Gate 7 is a configuration gate.
- **Tool trust.** Yosys, yosys-slang, the Sail back end, psgen, aig-manip and rIC3 are all in the trusted base. No proof certificates were produced; rIC3 can emit them, and doing so is the obvious next step for independent checking.
- **Silicon.** The RTL at a commit was verified. Nothing is claimed about the RP2350 die.

8 Process

The harness bring-up, the lemma discovery, the proof runs and the findings were carried out by an LLM agent (Claude, through Claude Code) driving the tools on the proof machine, with the author directing. This section records what that looked like, because it is the part of the method most likely to be reused.

Division of labour. The author chose the target, the configuration and the goals; decided each time whether to fix a divergence in the RTL or to weaken the comparison; decided to treat the multiplier as an axiom in the Ibex manner rather than spend days on an algebraic proof; and set the rules the agent ran under. The agent built the harness gate by gate, wrote every lemma, ran every round, read every counterexample, wrote the findings and this report's first draft. The author read the findings against the RTL and the specification text before accepting them.

Rules that held. No goal is reported proven without the chain checker's output, run by hand, copied verbatim. The author is notified only for a goal proven, a genuine divergence, or a decision to make. A running proof is never killed to make room; a round runs to its end. Every result is reported with the model manifest it came from. A running notebook (`RESUME.md`, about 1750 lines by the end) is kept current so that any session can pick up from the log alone; the notebook is not published, the log is.

What the agent was good at. Reading a counterexample to induction and naming the fact that rules it out. Almost every lemma came from one report: which harness predicates were false in the final frame, which pipeline signals changed. Turning a refutation into the right guard (a stall, a dead slot, a late fault) took one to three attempts. Running the loop unattended for hours, including at night, with the rules above.

What it was bad at, and what was done about it. Declaring victory. Both retractions of 18 September were the agent reporting a goal as proven on a chain it had not verified; the chain checker exists because of that, and the rule that no report is made without it. Losing track of the state after a strip, which is why the runner rather than the agent does the bookkeeping of what must be re-proven. Killing a stuck proof out of impatience; the tool permissions were set so that it could not.

Cost. The lemma loop took four days of wall-clock time on one machine; the bring-up before it passed its last three gates on 16 September. The Ibex proof it is modelled on was a multi-month effort by a specialist; that difference is mostly that the proof structure, the tools and the shape of every lemma were already known, and that a machine reading counterexample reports around the clock does not tire.

9 Related work

The Ibex proof [1] is the direct ancestor: the goals, the follower and ghost, the abstraction, the layered proof script, `psgen` and `aig-manip` are theirs, as is the decision to assume the M-type datapath. It proves about 1000 properties on a 32-core, 128 GB machine, with Jasper or with the open flow. Hazard3 ships its own riscv-formal [4] checks, which are bounded and per-instruction; the FENCE finding was caught upstream by the riscv-arch-test conformance suite, which enumerates encodings. The Sail RISC-V model [3] is the reference in both.

10 Conclusion

An RV32IMC, machine-mode configuration of Hazard3 v1.1.1, with five one-line fixes, is trace equivalent to the RISC-V Sail specification in architectural state and memory traffic, unboundedly, modulo the arithmetic of its multiplier/divider and the assumptions of section 7. The proof used only open tools and one machine for four days of lemma discovery. It found five conformance divergences in the core and one in the specification model, all reproduced independently. The multiplier axiom, the larger RP2350 configuration, liveness and proof certificates are the open items, in that order of difficulty.

Reproducibility

The repository holds the harness, the pinned commits and patches, the final proof script, the three proof logs, the chain verdicts, the tools, and the findings with their reproducers. `reproduce.sh` rebuilds both models from the pinned sources inside the Ibex formal Nix shell, re-proves every lemma with nothing stripped, and prints the 18 chain verdicts. It was run once on a fresh machine (a new 48-core instance holding only the Ibex checkout and its toolchain) on 20 September 2026: 265 of 265 lemmas re-proved, no refutation, all 18 goals proved by k-induction without an induction counterexample, and all 18 chains *proven modulo axioms [CtiWbMulDiv]*, in 3 h 40 min (`results/reproduce/`, `docs/REPRODUCE.md`). One base-layer lemma exceeded the 1800 s IC3 cap at the default seed and proved in 180 s at another; the script now retries a timeout with a second seed. `findings/upstream/repro/run.sh` reproduces the five Hazard3 findings on the CXXRTL testbench in minutes on a laptop.

References

[1] lowRISC and the University of Oxford. *End-to-End Formal Verification of Ibex Trace Equivalence against the Sail Model*, `ibex/dv/formal`, Louis-Emile Ploix, 2025.

[2] Luke Wren. *Hazard3: a 3-stage RV32IMACZb* processor*, `https://github.com/Wren6991/Hazard3`, v1.1.1.

[3] Alasdair Armstrong et al. *ISA Semantics for ARMv8-A, RISC-V, and CHERI-MIPS*, POPL 2019; `riscv/sail-riscv`.

[4] Claire Wolf. *riscv-formal*, `https://github.com/YosysHQ/riscv-formal`.